PocketWise

Personal Finance Tracker

DBMS Project Report

Course Name: Database Management Systems Course Code: UCS310

Submitted By:

Kasvi Bhatia -1024030796

Gurleen -1024030806

Kiesha Kapoor -1024031152

Degree: B.Tech (2nd Year) Department: Computer Science & Engineering

Instructor Name : Ms. Abhishelly

Academic Year: Jan–June (2025–26)

Declaration

We hereby declare that the project titled "**PocketWise — Personal Finance Tracker**" submitted according to the requirements for the course Database Management Systems (UCS310) is a record of original work carried out by us.

This project has been developed under the guidance of our course instructor **Ms. Abhishelly** and has not been submitted previously, either in part or in full, to any other institution or university for the award of any degree or diploma.

We confirm that all sources of information used in this project have been properly acknowledged and referenced. The work presented in this report is genuine and reflects our understanding and implementation of database concepts including data modelling, normalization, SQL queries, PL/SQL constructs, and system design.

We further declare that we have adhered to academic integrity and have not engaged in any form of plagiarism while preparing this report.

Date: _____ Signature: _____

Acknowledgement

We would like to express our sincere gratitude to our course instructor Ms. Abhishelly
for her valuable guidance, continuous support, and encouragement throughout the development of this project. Her insights and suggestions helped us understand the core concepts of Database Management Systems and apply them effectively in our work.

We are also thankful to the Department of Computer Science & Engineering, Thapar Institute of Engineering & Technology, for providing us with the necessary resources and a conducive learning environment to successfully complete this project.

We extend our heartfelt thanks to our friends and classmates for their cooperation, constructive feedback, and constant motivation during the course of this project. Their support played an important role in overcoming various challenges encountered during the implementation phase.

Finally, we would like to thank our family members for their unwavering support, patience, and encouragement, which helped us stay focused and complete this project successfully.

Abstract

PocketWise is a full-stack, database-driven web application designed to help individuals efficiently track and manage their personal finances. The system integrates income and expense tracking, custom category management, monthly budget setting, and automated financial summary generation into a single, centralized platform.

In today's fast-paced environment, managing personal finances manually is prone to errors, lacks real-time insight, and provides no automated reporting. PocketWise addresses these challenges by building a structured relational database backend in MySQL, connected to a Node.js and Express.js REST API. Users can register securely, log transactions, set budgets per spending category, and receive automated monthly summaries without any manual calculation.

The backend is implemented using MySQL, with Structured Query Language (SQL) used for all data definition, manipulation, and retrieval. Advanced database concepts including Entity-Relationship (ER) modelling, normalization up to Third Normal Form (3NF), triggers, stored procedures, cursors, and stored functions are incorporated to ensure data integrity, reduce redundancy, automate budget tracking, and generate financial analytics directly within the database layer.

Key highlights include three MySQL triggers that automate budget updates and maintain audit logs, two stored functions for computing user balances and category spending totals, and two stored procedures—one for monthly financial summaries and one using a cursor to generate category-wise breakdowns. JWT-based authentication ensures secure, per-user data isolation.

The proposed system provides a scalable, secure, and efficient solution for personal finance management. It reduces manual effort, minimizes calculation errors, and enhances data accuracy while demonstrating the practical implementation of advanced database management concepts in a real-world application.

Contents

1 Introduction

1.1 Background and Context

The management of personal finances involves handling large volumes of data related to income, expenses, budgets, and spending categories. Ensuring accuracy and consistency in such operations is essential, as even minor errors can lead to overspending, missed budget limits, or incorrect financial summaries. Traditional manual methods are inefficient and lack proper automation, analytics, and real-time tracking.

With the growing need for digital financial tools, managing transaction records, budget allocations, and spending analytics has become increasingly important. Without a centralized system, it becomes difficult to maintain consistent data, track spending in real time, and receive automated financial insights. Issues such as data redundancy, inconsistent calculations, and the absence of audit trails are common in manual approaches.

To address these challenges, PocketWise is designed as a centralized, database-driven solution that integrates all major personal finance operations into a single platform. It allows efficient management of transactions, budgets, and categories while ensuring data consistency and reliability through advanced MySQL features.

1.2 Scope of the Project

The scope of this project focuses on the design and implementation of a relational database system for the PocketWise Personal Finance Tracker. It includes the complete data modelling process, from developing an Entity-Relationship (ER) diagram to creating a normalized relational schema.

The project involves implementing the database using MySQL and applying SQL for data definition, manipulation, and retrieval. Advanced features such as triggers, stored procedures, cursors, and stored functions are used to automate operations like budget tracking, balance calculation, and monthly reporting while ensuring data integrity.

The system manages key components such as users, transactions, categories, and budgets within a centralized database. The primary focus of this report is on the backend database design and REST API, while the frontend is considered a supporting component and is not covered in detail.

2 .Problem Statement

The lack of a centralized system for managing personal finances leads to inefficiencies in tracking income, expenses, and budgets. Manual or disconnected systems increase the chances of errors, data inconsistency, and the absence of real-time financial summaries. As transaction volumes grow, it becomes difficult to maintain accurate and up-to-date records, making financial planning less reliable.

1.3 Limitations of the Manual or Traditional Approach

Currently, individuals managing finances through spreadsheets or manual record-keeping face several limitations. Important data related to income, expenses, and budgets is scattered, making it difficult to access and analyze. This leads to errors such as duplicate entries, incorrect balance calculations, and an inability to identify overspending.

There is no structured and searchable system to quickly retrieve or update records. The process is time-consuming and dependent on manual calculations, increasing the risk of mistakes. Key operations such as budget utilization tracking, monthly totals, and category-wise analysis must be done manually.

Another major limitation is the absence of automation. Without database triggers and stored procedures, operations such as updating a budget when a new expense is logged, or generating a monthly summary, require multiple manual steps that are prone to error and inconsistency.

1.4 Operational Challenges Without a Centralised System

Without a centralized system, managing personal finances becomes unorganized and error-prone. There is no standard mechanism for handling transaction data, budget allocation, and category relationships, which increases the risk of data inconsistency.

Furthermore, there is no automated mechanism to track spending against budget limits or to prevent budget overruns from going unnoticed. Similarly, when transactions are deleted, the corresponding budget totals are not automatically adjusted, leading to inaccurate data.

The absence of integration between different components also makes it impossible to generate reports such as monthly income versus expense summaries, category breakdowns, and all-time balance calculations automatically. These challenges highlight the need for a structured, database-driven Personal Finance Tracker that ensures accuracy, automation, and better financial oversight.

2 Objectives of the Project

The objectives of this project are to develop a structured and efficient database system for managing personal finances. The system aims to handle user accounts, transactions, spending categories, and budgets in an organized manner while ensuring accuracy, consistency, and reliability through advanced database features.

2.1 Entity-Relationship Data Modelling

The first objective is to design a comprehensive ER model for the system. This involves identifying key entities such as users, transactions, categories, budgets, and audit logs, along with their attributes and relationships. Proper definition of relationships and constraints ensures an accurate representation of real-world personal finance operations.

2.2 Conversion of ER Model to Relational Tables

The second objective is to convert the ER model into a relational schema. Each entity is mapped to a table, and relationships are implemented using primary and foreign keys, resulting in a structured and implementable database design.

2.3 Application of Normalization

The third objective is to normalize the database to eliminate redundancy and maintain data integrity. The schema is refined up to Third Normal Form (3NF), ensuring proper organization of data and improved consistency across the system.

2.4 SQL Implementation

The fourth objective is to implement the database using MySQL. This includes creating tables using DDL commands and managing data using DML operations. Various SQL queries involving joins, filtering, aggregation, and subqueries are used to efficiently retrieve and manage user financial data.

2.5 PL/SQL Constructs

The fifth objective is to incorporate procedural logic within the database using triggers, stored functions, stored procedures, and cursors. These components automate key operations like budget updates on transaction insert/delete, balance calculation, monthly summary generation, and category breakdown reporting.

2.6 Data Consistency Through Constraints and Transactions

The final objective is to ensure data integrity using constraints and transaction management. Primary keys, foreign keys, UNIQUE, NOT NULL, and ENUM constraints maintain accurate and valid data. Transaction control mechanisms ensure that operations execute reliably without partial updates or data inconsistency.

3 Proposed System

The proposed PocketWise system is a centralized, database-driven solution designed to overcome the limitations of manual personal finance management. The system provides a structured and efficient platform for recording transactions, managing spending categories, setting monthly budgets, and generating financial summaries, ensuring better organization, faster data retrieval, and improved overall accuracy.

The system is designed as a REST API-backed application supported by a robust relational database implemented using MySQL. It allows authenticated users to log income and expense transactions, create custom categories with icons and colors, define monthly budget limits per category, and retrieve automated financial summaries through dedicated API endpoints. All information is stored in a centralized database, enabling easy access, real-time updates, and consistent management of data.

One of the key features of the proposed system is its ability to maintain accurate and real-time budget records. When a new expense transaction is added, a MySQL trigger automatically increments the ‘spent’ column in the corresponding budget record. Similarly, when a transaction is deleted, a second trigger decrements the spent amount. This eliminates the need for manual budget updates and ensures data is always consistent.

The system also includes automated monthly financial summaries through a stored procedure that calculates total income, total expenses, and net balance for any given month. A second stored procedure uses a cursor to iterate through all spending categories and generates a detailed breakdown including budget limits and an over-budget flag for each category.

From a database perspective, the system is designed using Entity-Relationship modelling and normalized to eliminate redundancy and maintain data consistency. Relationships between entities are enforced using primary and foreign keys, ensuring referential integrity. JWT-based authentication ensures each user can only access their own data.

Overall, the proposed system improves financial awareness, reduces manual effort, and provides a reliable and scalable solution for personal finance management. By integrating all operations into a single platform with advanced database automation, it ensures better coordination, accuracy, and effective handling of financial data.

4 Entity-Relationship (E-R) Data Model

The Entity-Relationship (ER) model forms the conceptual foundation of the PocketWise system. It provides a structured and implementation-independent representation of the system by identifying key entities, their attributes, and the relationships between them. The model captures all major components of personal finance management.

4.1 Entities and Their Attributes

The Users entity represents registered individuals using the system. Each user is uniquely identified by a user_id and includes attributes such as name, email, password_hash, and created_at.

The Categories entity represents custom spending categories created by each user. It includes attributes: category_id, user_id (FK), name, icon, and color. Each category belongs to exactly one user.

The Transactions entity records all income and expense entries. It includes transaction_id, user_id (FK), category_id (FK), type (income or expense), amount, description, and transaction_date. It is the central entity in the system.

The Budgets entity stores monthly spending limits per category per user. It includes budget_id, user_id (FK), category_id (FK), amount (limit), spent (auto-updated by trigger), month, and year.

The User_Deletion_Log entity is a weak/audit entity that records details of deleted user accounts. It is populated automatically by a BEFORE DELETE trigger on the Users table. Attributes include log_id, user_id, email, name, deleted_at, and deleted_by.

4.2 Relationships and Cardinality

The relationship between Users and Categories is one-to-many: one user can create multiple categories, but each category belongs to exactly one user.

The relationship between Users and Transactions is one-to-many: one user can have many transactions, but each transaction belongs to one user.

The relationship between Categories and Transactions is one-to-many: one category can be associated with multiple transactions, while each transaction optionally references one category.

The relationship between Users and Budgets is one-to-many: one user can set multiple budgets (one per category per month), but each budget belongs to one user.

The relationship between Categories and Budgets is one-to-many: one category can have budgets set for multiple months, while each budget references one category.

The relationship between Users and User_Deletion_Log is one-to-one in effect: when a user is deleted, exactly one log record is created capturing their details before deletion.

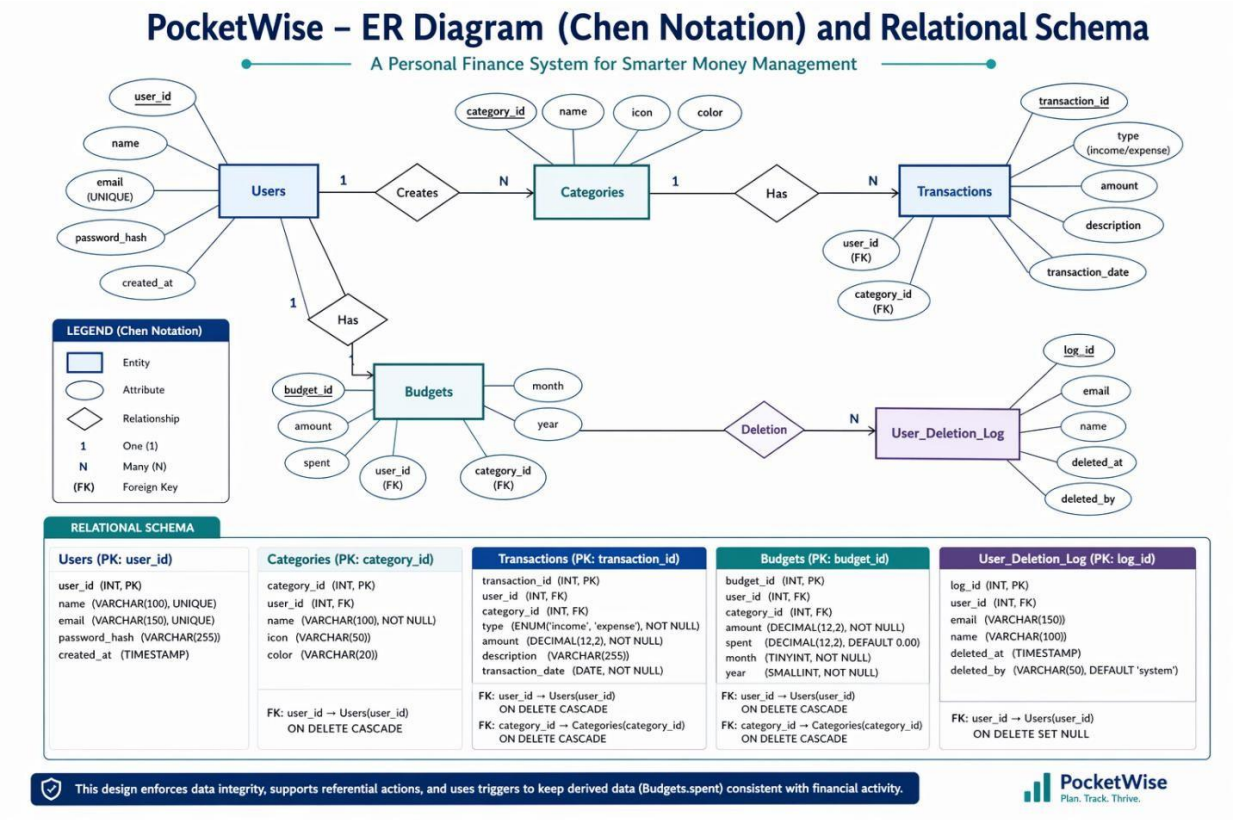
4.3 E-R Diagram Summary

The system can be summarized as follows:

- Users (1:N) Categories
- Users (1:N) Transactions
- Categories (1:N) Transactions
- Users (1:N) Budgets
- Categories (1:N) Budgets
- Transactions → triggers → Budgets (via trg_after_transaction_insert and trg_after_transaction_delete)
- Users → triggers → User_Deletion_Log (via trg_before_user_delete)

This structured design ensures that all relationships are clearly defined and efficiently mapped into relational tables, supporting accurate and consistent data management within the PocketWise system.

E-R Diagram


PocketWise – ER Diagram (Chen Notation) and Relational Schema
A Personal Finance System for Smarter Money Management
Users
Categories
Transactions
Budgets
User_Deletion_Log
Creates
Has
Has
Deletion
user_id
name
email (UNIQUE)
password_hash
created_at
category_id
name
icon
color
transaction_id
type (income/expense)
amount
description
transaction_date
user_id (FK)
category_id (FK)
budget_id
amount
spent
user_id (FK)
category_id (FK)
month
year
log_id
email
name
deleted_at
deleted_by
LEGEND (Chen Notation)
Entity
Attribute
Relationship
1 One (1)
N Many (N)
(FK) Foreign Key
RELATIONAL SCHEMA
Users (PK: user_id)
user_id (INT, PK)
name (VARCHAR(100), UNIQUE)
email (VARCHAR(150), UNIQUE)
password_hash (VARCHAR(255))
created_at (TIMESTAMP)
Categories (PK: category_id)
category_id (INT, PK)
user_id (INT, FK)
name (VARCHAR(100), NOT NULL)
icon (VARCHAR(50))
color (VARCHAR(20))
FK: user_id → Users(user_id) ON DELETE CASCADE
Transactions (PK: transaction_id)
transaction_id (INT, PK)
user_id (INT, FK)
category_id (INT, FK)
type (ENUM('income', 'expense'), NOT NULL)
amount (DECIMAL(12,2), NOT NULL)
description (VARCHAR(255))
transaction_date (DATE, NOT NULL)
FK: user_id → Users(user_id) ON DELETE CASCADE
FK: category_id → Categories(category_id) ON DELETE CASCADE
Budgets (PK: budget_id)
budget_id (INT, PK)
user_id (INT, FK)
category_id (INT, FK)
amount (DECIMAL(12,2), NOT NULL)
spent (DECIMAL(12,2), DEFAULT 0.00)
month (TINYINT, NOT NULL)
year (SMALLINT, NOT NULL)
FK: user_id → Users(user_id) ON DELETE CASCADE
FK: category_id → Categories(category_id) ON DELETE CASCADE
User_Deletion_Log (PK: log_id)
log_id (INT, PK)
user_id (INT, FK)
email (VARCHAR(150))
name (VARCHAR(100))
deleted_at (TIMESTAMP)
deleted_by (VARCHAR(50), DEFAULT 'system')
FK: user_id → Users(user_id) ON DELETE SET NULL
This design enforces data integrity, supports referential actions, and uses triggers to keep derived data (Budgets.spent) consistent with financial activity.
PocketWise
Plan. Track. Thrive.

5 Conversion of E-R Model to Relational Tables

The ER model is converted into a relational schema using standard mapping techniques. Each entity is transformed into a table, and relationships are represented using primary and foreign keys to ensure data integrity and consistency across the system.

5.1 Users Table

The Users table stores registered user accounts. Each user is uniquely identified by a user_id.

```
users (
    user_id            INT AUTO_INCREMENT PRIMARY
    KEY, name          VARCHAR(100)  NOT NULL,
    email              VARCHAR(150)  NOT NULL UNIQUE,
    password_hash VARCHAR(255)  NOT NULL,
    created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Categories Table

The Categories table stores custom spending categories created by each user.

```
categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id          INT          NOT NULL,
    name             VARCHAR(100) NOT NULL,
    icon             VARCHAR(50),
    color            VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);
```

5.3 Transactions Table

The Transactions table records all income and expense entries for each user.

```
transactions (
    transaction_id        INT AUTO_INCREMENT PRIMARY
    KEY, user_id          INT NOT NULL,
    category_id           INT,
    type                  ENUM('income','expense') NOT NULL,
    amount                DECIMAL(12,2) NOT NULL,
    description
                          VARCHAR(2
    55), transaction_date DATE NOT
    NULL,
    FOREIGN KEY (user_id)          REFERENCES users(user_id) ON DELETE
    CASCADE, FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

5.4 Budgets Table

The Budgets table stores monthly spending limits per category. The spent column is auto-updated by triggers.

```
budgets (
    budget_id        INT AUTO_INCREMENT PRIMARY KEY,
    user_id          INT           NOT NULL,
    category_id INT  NOT NULL, amount
                     DECIMAL(12,2) NOT
    NULL, spent      DECIMAL(12,2) DEFAULT
    0.00, month      TINYINT       NOT
    NULL, year       SMALLINT      NOT
    NULL,
    FOREIGN KEY (user_id)          REFERENCES users(user_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

5.5 User Deletion Log Table (Audit / Weak Entity)

The user_deletion_log table acts as a permanent audit record. It is populated by a BEFORE DELETE trigger on the users table.

```
user_deletion_log (
    log_id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id         INT          NOT
    NULL, email     VARCHAR(150),
    name            VARCHAR(100),
    deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    deleted_by VARCHAR(50)  DEFAULT 'system'
);
```

6 Database Design

The database design for PocketWise is developed using a structured approach that includes conceptual, logical, and physical design phases. This ensures that the system is efficient, scalable, and capable of handling complex personal finance operations.

The conceptual design is represented using an Entity-Relationship (ER) model, which identifies key entities such as users, transactions, categories, budgets, and audit logs, along with their attributes and relationships. This model provides a clear understanding of how different components of the system interact.

The logical design phase involves converting the ER model into a relational schema. Each entity is mapped to a table, and relationships are implemented using primary and foreign keys. The schema is further refined through normalization to eliminate redundancy and ensure data consistency.

In the physical design phase, the database is implemented in MySQL. Constraints such as primary keys, foreign keys, NOT NULL, UNIQUE, and ENUM are applied to maintain data integrity. Advanced features such as triggers, stored procedures, cursors, and stored functions automate operations such as budget updates, balance calculation, and monthly reporting.

Tier	Component	Technology	Role
Conceptual	ER Model	Chen Notation	Entities, attributes, relationships
Logical	Relational Schema	Normalization to 3NF	Tables, keys, dependencies
Physical	MySQL Database	MySQL 8.0	DDL, DML, triggers, procedures
Application	REST API	Node.js + Express.js	Business logic, routing
Security	Authentication	JWT + bcryptjs	Token-based user isolation

Overall, the database design ensures accurate data management, efficient retrieval, and reliable operation of the PocketWise system, making it suitable for real-world personal finance applications.

7 Normalization — Up to 3NF and BCNF

Normalization is a systematic process used in relational database design to reduce redundancy and improve data consistency. In PocketWise, the database is divided into separate tables for users, categories, transactions, budgets, and audit logs. This structured separation ensures that each table stores only directly related data, avoiding duplication and improving maintainability.

7.1 First Normal Form (1NF)

A table is in 1NF if all attributes contain atomic values and there are no repeating groups or multi-valued attributes.

In PocketWise, all tables satisfy 1NF because:

- All column values are atomic (e.g., name, email, amount, month, year are single-valued)
- No column stores multiple values or arrays
- Each table has a clearly defined primary key (user_id, category_id, transaction_id, budget_id, log_id)
- The type column in transactions uses ENUM to restrict to a single atomic value per row

7.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and all non-key attributes are fully dependent on the primary key (no partial dependency). In PocketWise, all tables use a single-column auto-increment primary key, eliminating partial dependency entirely.

In the transactions table, all attributes (user_id, category_id, type, amount, description, transaction_date) depend fully on transaction_id. In the budgets table, all attributes (user_id, category_id, amount, spent, month, year) depend fully on budget_id. Hence, all tables satisfy 2NF.

7.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and there are no transitive dependencies (no non-key attribute depends on another non-key attribute). In PocketWise:

- The transactions table stores only foreign keys (user_id, category_id) rather than duplicating user name or category name
- The budgets table stores only category_id, not category name or icon, avoiding transitive dependency
- All user information is stored solely in the users table, not repeated elsewhere
- All category information (name, icon, color) is stored solely in the categories table

This design removes redundancy, ensures consistency, and satisfies 3NF for all tables.

7.4 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every functional dependency $X \rightarrow Y$, X is a superkey. All tables in PocketWise satisfy BCNF because each table has a well-defined single-column primary key, no non-key attribute determines another attribute, and there are no overlapping candidate keys that could violate BCNF.

7.5 Normalization Summary

Table Name	1NF	2NF	3NF	BCNF	Justification
users	Yes	Yes	Yes	Yes	All attributes depend on user_id
categories	Yes	Yes	Yes	Yes	Category details depend on category_id
transactions	Yes	Yes	Yes	Yes	Transaction details depend on transaction_id
budgets	Yes	Yes	Yes	Yes	Budget details depend on budget_id

user_deletion_log	Yes	Yes	Yes	Yes	Log details depend on log_id

8 SQL Implementation and Queries

After designing and normalizing the database, PocketWise was implemented in MySQL using SQL. The database contains tables for users, categories, transactions, budgets, and audit logs. DDL commands were used to create tables, while DML commands insert and manage data. The project also includes views and queries for transaction retrieval, budget analysis, monthly summaries, and top spending reports.

8.1 Retrieving All Transactions for a User

```
SELECT t.transaction_id, t.type, t.amount, t.description,
       t.transaction_date, c.name AS category, c.icon, c.color
FROM    transactions t
LEFT JOIN categories c ON t.category_id = c.category_id
WHERE  t.user_id = ?
ORDER  BY t.transaction_date DESC;
```

8.2 Monthly Income and Expense Summary

```
SELECT
  COALESCE(SUM(CASE WHEN type = 'income'  THEN amount ELSE 0 END), 0) AS total_income,
  COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0 END), 0) AS total_expense,
  COALESCE(SUM(CASE WHEN type = 'income'  THEN amount ELSE 0 END), 0) -
  COALESCE(SUM(CASE WHEN type = 'expense' THEN amount ELSE 0 END), 0) AS net_balance
FROM transactions
WHERE user_id = ? AND MONTH(transaction_date) = ? AND YEAR(transaction_date) = ?;
```

8.3 Budget Utilization Report

```
SELECT b.budget_id, c.name AS category, c.icon, c.color,
       b.amount AS budget_limit, b.spent,
       IF(b.spent > b.amount, 1, 0) AS over_budget
FROM    budgets b
JOIN    categories c ON b.category_id =
c.category_id WHERE  b.user_id = ? AND b.month = ?
AND b.year = ? ORDER  BY b.spent DESC;
```

8.4 Top Spending Categories (Aggregate Query)

```
SELECT c.name AS category, c.icon,
       COUNT(t.transaction_id) AS transaction_count,
       SUM(t.amount) AS total_spent
FROM    transactions t
JOIN    categories c ON t.category_id =
c.category_id WHERE  t.user_id = ? AND t.type =
'expense'
  AND  MONTH(t.transaction_date) = ? AND YEAR(t.transaction_date) = ?
GROUP  BY c.category_id, c.name, c.icon
ORDER  BY total_spent DESC;
```

8.5 Date Range Filtering and Activity Analysis

```
SELECT DATE(transaction_date) AS txn_day,
       COUNT(*) AS total_transactions,
       SUM(CASE WHEN type='income'  THEN amount ELSE 0 END) AS income,
       SUM(CASE WHEN type='expense' THEN amount ELSE 0 END) AS expenses
FROM    transactions
WHERE  user_id = ? AND transaction_date BETWEEN ? AND ?
GROUP  BY DATE(transaction_date)
ORDER  BY txn_day DESC;
```

9b PL/SQL Constructs

Procedural extensions to SQL, supported in MySQL through its stored program framework, enable the implementation of complex business logic directly within the database layer. This reduces dependency on application-level code, improves performance, and ensures consistency by centralizing logic. The following constructs are implemented in PocketWise.

8.6 Trigger 1 — Auto-Update Budget on Expense Insert

This trigger fires AFTER INSERT on transactions. If the new transaction is an expense with a category, it automatically increments the spent column in the matching budget record.

```
DELIMITER $$
CREATE TRIGGER trg_after_transaction_insert
AFTER INSERT ON transactions FOR EACH ROW
BEGIN
    IF NEW.type = 'expense' AND NEW.category_id IS NOT NULL THEN
        UPDATE budgets
        SET             spent = spent + NEW.amount
        WHERE  user_id = NEW.user_id AND category_id = NEW.category_id
          AND  month = MONTH(NEW.transaction_date)
          AND  year  = YEAR(NEW.transaction_date);
    END IF;
END$$
DELIMITER ;
```

8.7 Trigger 2 — Reverse Budget on Expense Delete

This trigger fires AFTER DELETE on transactions and decrements the spent column in budgets, using GREATEST(0, ...) to prevent negative values.

```
DELIMITER $$
CREATE TRIGGER trg_after_transaction_delete
AFTER DELETE ON transactions FOR EACH ROW
BEGIN
    IF OLD.type = 'expense' AND OLD.category_id IS NOT NULL THEN
        UPDATE budgets
        SET             spent = GREATEST(0, spent - OLD.amount)
        WHERE  user_id = OLD.user_id AND category_id = OLD.category_id
          AND  month = MONTH(OLD.transaction_date)
          AND  year  = YEAR(OLD.transaction_date);
    END IF;
END$$
DELIMITER ;
```

8.8 Trigger 3 — Audit Log Before User Deletion

This BEFORE DELETE trigger on users captures the user's details into the audit table before the record is permanently removed.

```
DELIMITER $$
CREATE TRIGGER trg_before_user_delete
BEFORE DELETE ON users FOR EACH ROW
BEGIN
    INSERT INTO user_deletion_log (user_id, email, name)
    VALUES (OLD.user_id, OLD.email, OLD.name);
END$$
DELIMITER ;
```

8.9 Stored Function — Get User Balance

This stored function returns the all-time net balance (total income minus total expenses) for a given user.

```
DELIMITER $$
CREATE FUNCTION get_user_balance(p_user_id INT)
```

```
RETURNS DECIMAL(12,2) DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_income  DECIMAL(12,2) DEFAULT 0.00;
    DECLARE v_expense DECIMAL(12,2) DEFAULT 0.00;
    SELECT COALESCE(SUM(amount),0) INTO v_income  FROM transactions
    WHERE  user_id = p_user_id AND type = 'income';
    SELECT COALESCE(SUM(amount),0) INTO v_expense FROM transactions
    WHERE  user_id = p_user_id AND type = 'expense';
    RETURN v_income - v_expense;
END$$
DELIMITER ;
```

8.10 Stored Procedure — Monthly Summary

This procedure returns total income, total expenses, and net balance for a specified month and year. It is called directly from the /api/summary backend route.

```
DELIMITER $$
CREATE PROCEDURE get_monthly_summary(
    IN p_user_id INT, IN p_month TINYINT, IN p_year SMALLINT)
BEGIN
    SELECT
      COALESCE(SUM(CASE WHEN type='income'  THEN amount ELSE 0 END),0) AS total_income,
      COALESCE(SUM(CASE WHEN type='expense' THEN amount ELSE 0 END),0) AS total_expense,
      COALESCE(SUM(CASE WHEN type='income'  THEN amount ELSE 0 END),0) -
      COALESCE(SUM(CASE WHEN type='expense' THEN amount ELSE 0 END),0) AS net_balance
    FROM transactions
    WHERE user_id=p_user_id
      AND MONTH(transaction_date)=p_month
      AND YEAR(transaction_date)=p_year;
END$$
DELIMITER ;
```

8.11 Cursor — Category Breakdown Processing

This procedure uses a cursor to iterate through every category in which the user spent money during the given month, computing totals and comparing against budget limits.

```
DELIMITER $$
CREATE PROCEDURE get_category_breakdown(
    IN p_user_id INT, IN p_month TINYINT, IN p_year SMALLINT)
BEGIN
    DECLARE v_done             INT DEFAULT FALSE;
    DECLARE v_cat_id           INT;
    DECLARE v_cat_name
                               VARCHAR(1
    00); DECLARE v_total_spent
    DECIMAL(12,2); DECLARE v_budget_lim
    DECIMAL(12,2); DECLARE cur CURSOR
    FOR
        SELECT DISTINCT c.category_id, c.name
        FROM transactions t JOIN categories c ON t.category_id=c.category_id
        WHERE t.user_id=p_user_id AND t.type='expense'
          AND MONTH(t.transaction_date)=p_month
          AND YEAR(t.transaction_date)=p_year;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done=TRUE;
    DROP TEMPORARY TABLE IF EXISTS tmp_breakdown;
    CREATE TEMPORARY TABLE tmp_breakdown(
        category_name VARCHAR(100), total_spent DECIMAL(12,2),
        budget_limit DECIMAL(12,2), over_budget TINYINT(1));
    OPEN cur;
    cat_loop: LOOP
        FETCH cur INTO v_cat_id, v_cat_name;
        IF v_done THEN LEAVE cat_loop; END IF;
        SET v_total_spent = get_category_total_spent(
            p_user_id, v_cat_id, p_month, p_year);
        SELECT COALESCE(amount,0) INTO v_budget_lim FROM budgets
        WHERE user_id=p_user_id AND category_id=v_cat_id
          AND month=p_month AND year=p_year LIMIT 1;
        INSERT INTO tmp_breakdown VALUES(
```

```
        v_cat_name, v_total_spent, v_budget_lim,
        IF(v_budget_lim>0 AND v_total_spent>v_budget_lim,1,0));
END LOOP;
CLOSE cur;
SELECT * FROM tmp_breakdown ORDER BY total_spent DESC;
DROP TEMPORARY TABLE IF EXISTS tmp_breakdown;
      END$$ DELIMITER ;
```

9 Data Consistency — Constraints and Transactions

Ensuring data accuracy and consistency is a critical aspect of database design. This is achieved using integrity constraints and transaction management, which together ensure reliable and valid data operations within the PocketWise system.

9.1 Integrity Constraints

Constraint	Applied To	Purpose
PRIMARY KEY	All tables	Unique identification of each record
FOREIGN KEY	categories, transactions, budgets	Maintains referential integrity between tables
NOT NULL	users, transactions, budgets	Prevents missing or null critical values
UNIQUE	email in users	Prevents duplicate user accounts
ENUM	type in transactions	Restricts values to 'income' or 'expense' only
DEFAULT	spent in budgets, created_at in users	Ensures safe default values on insert
ON DELETE CASCADE	categories, transactions FK to users	Auto-removes dependent records on user deletion

9.2 Transaction Management

Transactions ensure operations are atomic and consistent. The example below shows how budget update and transaction insertion are wrapped in a transaction to prevent partial data updates.

```
START TRANSACTION;

INSERT INTO transactions (user_id, category_id, type, amount,
    description, transaction_date)
VALUES (1, 3, 'expense', 1500.00, 'Grocery shopping', '2025-04-15');

-- Trigger trg_after_transaction_insert fires automatically here
-- No manual UPDATE to budgets is needed

COMMIT;

-- On error:
ROLLBACK;
```

This ensures data consistency and prevents partial updates. The trigger mechanism further guarantees that budget updates are atomic with the transaction insert, as both occur within the same database transaction context.

10Sample Data and Testing

To validate the correctness of the database schema and demonstrate the functionality of the system, representative sample data was inserted into the database. This data covers key use cases such as user registration, transaction logging, budget setting, trigger execution, and procedure calling.

10.1User Records

user_id	name	email	created_at
1	Rumani	rumani@example.com	2025-01-10
2	Yashika	yashika@example.com	2025-01-12
3	Aryan Bansal	aryan@example.com	2025-01-15

10.2Categories and Transactions

transaction_id	user_id	category	type	amount	date
1	1	Food	expense	850.00	2025-04-10
2	1	Salary	income	45000.00	2025-04-01
3	2	Transport	expense	300.00	2025-04-12
4	3	Food	expense	1200.00	2025-04-15
5	1	Entertainment	expense	600.00	2025-04-20

10.3Budget Records (After Trigger Execution)

budget_id	user_id	category	amount (limit)	spent (auto)	over_budget
1	1	Food	1000.00	850.00	No
2	1	Entertainment	500.00	600.00	YES
3	2	Transport	400.00	300.00	No

10.4Testing and Validation

The system was tested for the following scenarios:

- Correct primary and foreign key enforcement
- Proper execution of SQL queries with joins and aggregations
- Trigger execution: budget spent column auto-updated on INSERT and DELETE
- Stored procedure: get_monthly_summary returns correct income/expense/balance
- Cursor procedure: get_category_breakdown returns per-category totals with over_budget flag
- Stored function: get_user_balance returns correct all-time balance
- Audit trigger: user_deletion_log populated before user DELETE
- JWT authentication: unauthorized requests correctly rejected with 401

✓ All outputs were correct and consistent with expected results.

11Tools and Technologies Used

The development of PocketWise uses a combination of database, backend, and development technologies to ensure efficient data management and smooth processing of operations.

Category	Technology	Purpose
Database	MySQL 8.0	Stores and manages structured data for users, transactions, categories, and budgets
Query Language	SQL	Used for data definition (DDL), manipulation (DML), and retrieval
Procedural Logic	MySQL PL/SQL	Implements triggers, stored procedures, cursors, and functions for automation
Runtime	Node.js v16+	Server-side JavaScript runtime for the backend application
Web Framework	Express.js v4.18	REST API routing, middleware management, and request handling
DB Driver	mysql2 v3.6	Node.js MySQL client with promise support for async DB operations
Authentication	jsonwebtoken v9	JWT token generation and verification for secure API access
Password Security	bcryptjs v2.4	Hashing and salt-based password storage
Configuration	dotenv v16	Environment variable management for DB credentials and secrets
Dev Tool	nodemon v3	Auto-restart server during development on file changes
IDE	VS Code	Code editing, Git integration, and debugging
DB Management	MySQL Workbench	Database schema design, query execution, and management
Version Control	Git / GitHub	Source code versioning and team collaboration

12Actual Outcomes

The PocketWise Personal Finance Tracker was successfully implemented as a database-driven application for managing personal income, expenses, budgets, and financial summaries. A relational database was designed using an Entity-Relationship (ER) model and converted into a normalized schema up to Third Normal Form (3NF), ensuring minimal redundancy and high data integrity.

All required tables were created in MySQL with appropriate primary keys, foreign keys, NOT NULL, UNIQUE, ENUM, and ON DELETE CASCADE constraints to maintain relationships between users, categories, transactions, and budgets. SQL queries were implemented to perform data insertion, retrieval, updating, and deletion efficiently. Complex queries involving multi-table joins, conditional aggregation, date-based filtering, and subqueries were successfully executed.

Three MySQL triggers were implemented and verified to function correctly: the AFTER INSERT trigger auto-updates budget spent on new expense transactions, the AFTER DELETE trigger reverses the spent amount when a transaction is removed, and the BEFORE DELETE trigger records user details in the audit log before account deletion.

Two stored functions were implemented: get_user_balance returns a user's all-time net financial position, and get_category_total_spent computes monthly spending per category and is reused inside the cursor procedure. Two stored procedures were implemented and tested: get_monthly_summary delivers a single-row income/expense/balance report, and get_category_breakdown uses a cursor to generate a complete category-wise breakdown with budget comparison and over_budget flags.

JWT-based authentication was implemented and verified to correctly reject unauthorized requests with a 401 response. The system was tested using sample data inserted across all tables, and all functionalities performed as expected. The results demonstrate that the system is efficient, reliable, and capable of handling real-world personal finance management tasks effectively.

13Conclusion

The PocketWise Personal Finance Tracker successfully demonstrates the practical application of database management concepts in solving real-world personal finance challenges. The project addresses the limitations of manual and unstructured financial record-keeping by providing a centralized, database-driven solution for managing transactions, budgets, categories, and financial summaries.

A well-structured Entity-Relationship (ER) model was designed and systematically converted into a relational schema. The database was normalized up to Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF), ensuring minimal redundancy and high data integrity. The implementation using MySQL, along with comprehensive SQL queries, enabled efficient data storage, retrieval, and manipulation.

Advanced database features were the key highlight of this project. Three triggers automate budget tracking and audit logging without any application-level intervention. Two stored functions provide reusable, testable computation for user balance and category spending. Two stored procedures—one simple and one using a cursor—deliver automated monthly reports and detailed category breakdowns directly from the database layer.

The REST API backend built with Node.js and Express.js demonstrates how a modern web application can leverage advanced database capabilities to minimize application code, improve reliability, and deliver consistent results. JWT-based authentication ensures secure, per-user data isolation throughout the system.

The system was tested comprehensively using sample data, and all functionalities performed as expected. Overall, the project highlights the importance of a well-designed database system in improving efficiency, ensuring data consistency, and providing a reliable solution for personal financial management.

14Future Scope

The PocketWise system can be further enhanced by incorporating additional features to improve its functionality, usability, and scalability.

One important enhancement is the development of a React.js frontend dashboard with interactive charts and graphs, allowing users to visualize their spending patterns, income trends, and budget health in real time. This would significantly improve the user experience over the current API-only interface.

The system can be enhanced by integrating automated email and SMS notifications that alert users when their spending approaches or exceeds a set budget limit for any category. This proactive feature would help users take corrective action before overspending occurs.

Another valuable addition is support for recurring transactions, enabling users to define subscriptions, EMIs, or regular income entries that are automatically logged at defined intervals without manual entry each time.

Financial goal setting and savings tracking modules can be introduced, allowing users to define savings targets (such as a vacation fund or emergency corpus) and track their progress through automated balance allocations.

The system can be extended with multi-currency support and real-time exchange rate integration, making it useful for users who earn or spend in multiple currencies.

Machine learning-based spending prediction and anomaly detection can be incorporated to flag unusual spending patterns and provide personalized financial recommendations based on historical data.

Finally, deploying the system on a cloud platform such as AWS, GCP, or Heroku with automated database backups, horizontal scaling, and a React Native mobile application consuming the existing REST API would transform PocketWise into a fully production-ready personal finance solution.

15References

The following resources were referred to during the design and implementation of the PocketWise Personal Finance Tracker:

- Elmasri, R., & Navathe, S. B., Fundamentals of Database Systems, Pearson Education.
- Silberschatz, A., Korth, H. F., & Sudarshan, S., Database System Concepts, McGraw-Hill.
- MySQL Documentation — https://dev.mysql.com/doc/
- MySQL Stored Procedures & Triggers — https://dev.mysql.com/doc/refman/8.0/en/stored-programs-views.html
- W3Schools SQL Tutorial — https://www.w3schools.com/sql/
- GeeksforGeeks – Database Management System — https://www.geeksforgeeks.org/dbms/
- Express.js Documentation — https://expressjs.com/
- Node.js Documentation — https://nodejs.org/en/docs/
- JSON Web Tokens (JWT) — https://jwt.io/
- npm: mysql2 Package — https://www.npmjs.com/package/mysql2
- npm: bcryptjs Package — https://www.npmjs.com/package/bcryptjs
- GitHub Repository —https://github.com/kieshaaaa/Pocketwise-main